Week-3: Generating and Solving Uniform Random 3-SAT

Varsha(20261604007), Rahul Kumar(20261602015), Sukriti Singh(20261602019)

November 1, 2026

I PROBLEM STATEMENT

Write programs to generate uniform random k-SAT instances and, for $k = 3$, solve a set of random 3-SAT problems for different (m, n) combinations. Compare Hill-Climbing, Beam-Search (beam widths 3 and 4), and Variable-Neighborhood Descent (three neighborhoods). Use two heuristic functions and compare them with respect to penetrance (fraction of runs solved).

A UNIFORM RANDOM K-SAT (FIXED CLAUSE LENGTH MODEL)

Given k, m, n, generate m clauses. Each clause picks k distinct variables from $\{x_1 ... x_n\}$ uniformly at random; each literal is negated with probability 1/2. For $k = 3$ we obtain uniform random 3-SAT.

Each variable x_i appears as positive literal x_i or negative literal $\neg x_i$ with equal probability. The instance is represented as a list of clauses, where each clause is a disjunction of k literals.

II PYTHON CODE

A SIMPLE K-SAT CODE GENERATOR

```
import random

def generate_kSAT(k, m, n):
    clauses = []
    for _ in range(m):
        # pick k distinct variables
        vars_chosen = random.sample(range(1, n+1), k)
        clause = []
        for v in vars_chosen:
            if random.choice([True, False]):
                clause.append(f"x{v}")
            else:
                clause.append(f" x {v}")
```

```
        clauses.append(clause)
    return clauses

# Example usage
k = 3   # length of each clause
m = 5   # number of clauses
n = 4   # number of variables

formula = generate_kSAT(k, m, n)

print("Random k-SAT formula:")
for i, clause in enumerate(formula, 1):
    print(f"C{i}: ({' v '.join(clause)})")
```

B 3-SAT (HC, BEAM=3/4, VND) WITH TWO HEURISTICS

```
import random

# 3-SAT Generator
def generate_3sat(m, n):
    """Generate random 3-SAT instance with m clauses, n variables
       """
    clauses = []
    for _ in range(m):
        vars3 = random.sample(range(1, n+1), 3)
        clause = []
        for v in vars3:
            if random.choice([True, False]):
                clause.append(v)
            else:
                clause.append(-v)
        clauses.append(clause)
    return clauses

def evaluate(clauses, assignment):
    """Number of satisfied clauses"""
    count = 0
    for c in clauses:
        ok = False
        for lit in c:
            v = abs(lit)
            val = assignment[v]
            if (lit > 0 and val) or (lit < 0 and not val):
                ok = True
        if ok: count += 1
    return count

# two heuristics
def h1(clauses, A): return evaluate(clauses, A)
def h2(clauses, A): return 2*evaluate(clauses, A) - len(clauses)
```

```
34
35  # hill climbing
36  def hill_climbing(clauses, n, heuristic, steps=1000):
37      A = [None] + [random.choice([False, True]) for _ in range(n)]
38      for _ in range(steps):
39          if evaluate(clauses, A) == len(clauses):
40              return True
41          best_score, best_var = heuristic(clauses, A), None
42          for v in range(1, n+1):
43              A[v] = not A[v]
44              score = heuristic(clauses, A)
45              A[v] = not A[v]
46              if score > best_score:
47                  best_score, best_var = score, v
48          if best_var is None:
49              v = random.randint(1, n)
50              A[v] = not A[v]
51          else:
52              A[best_var] = not A[best_var]
53      return False
54
55  def beam_search(clauses, n, heuristic, beam_width=3, steps=200):
56      beam = []
57      for _ in range(beam_width):
58          A = [None] + [random.choice([False, True]) for _ in range
                (n)]
59          beam.append(A)
60      for _ in range(steps):
61          new_beam = []
62          for A in beam:
63              if evaluate(clauses, A) == len(clauses): return True
64              for v in range(1, n+1):
65                  B = A.copy()
66                  B[v] = not B[v]
67                  new_beam.append(B)
68          new_beam.sort(key=lambda x: heuristic(clauses, x),
                reverse=True)
69          beam = new_beam[:beam_width]
70      return False
71
72  def vnd(clauses, n, heuristic, steps=1000):
73      A = [None] + [random.choice([False, True]) for _ in range(n)]
74      for _ in range(steps):
75          if evaluate(clauses, A) == len(clauses): return True
76          improved = False
77          curr_score = heuristic(clauses, A)
78          for v in range(1, n+1):
79              A[v] = not A[v]
80              if heuristic(clauses, A) > curr_score:
81                  improved = True
82                  curr_score = heuristic(clauses, A)
```

```
            else:
                A[v] = not A[v]
        if not improved:
            v = random.randint(1, n)
            A[v] = not A[v]
    return False

n = 6   # variables
m = 15  # clauses
clauses = generate_3sat(m, n)
print("Generated 3-SAT clauses:", clauses)

for hname, hfun in [("h1", h1), ("h2", h2)]:
    print(f"\nUsing heuristic {hname}:")
    print(" Hill-Climbing:", hill_climbing(clauses, n, hfun))
    print(" Beam Search (width=3):", beam_search(clauses, n, hfun
        , beam_width=3))
    print(" Beam Search (width=4):", beam_search(clauses, n, hfun
        , beam_width=4))
    print(" VND:", vnd(clauses, n, hfun))
```

III HOW TO RUN

The codes were executed using Python 3.14 in Microsoft VS Code.

For k-SAT generation:

```
python k-sat-problem.py
```

For 3-SAT solving with heuristics:

```
python 3-sat-problem.py
```

The program generates random clauses, evaluates them using two heuristics h1 and h2, and applies Hill-Climbing, Beam Search (width 3 and 4), and VND to check satisfiability.

IV SCREENSHOTS OF RESULTS

A K-SAT.JPG

Random k-SAT formula for k=3, m=5, n=4. Each clause contains disjunction of 3 random literals.

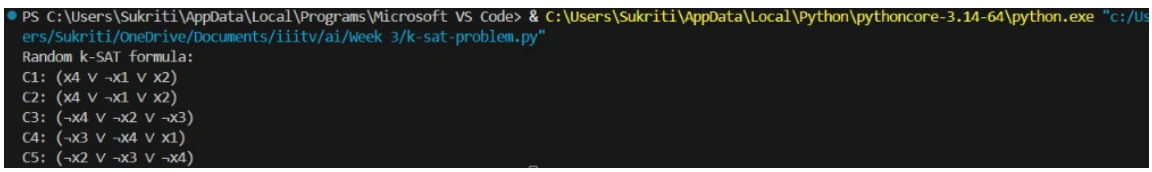


Figure 1: Output of Simple k-SAT Generator - 5 clauses with 4 variables

Output in text:

```
C1: (x4 v ~x1 v x2)
C2: (x4 v ~x1 v x2)
C3: (~x4 v ~x2 v ~x3)
C4: (~x3 v ~x4 v x1)
C5: (~x2 v ~x3 v ~x4)
```

B 3-SAT.JPG

3-SAT solving for m=15 clauses, n=6 variables using h1 and h2 heuristics.

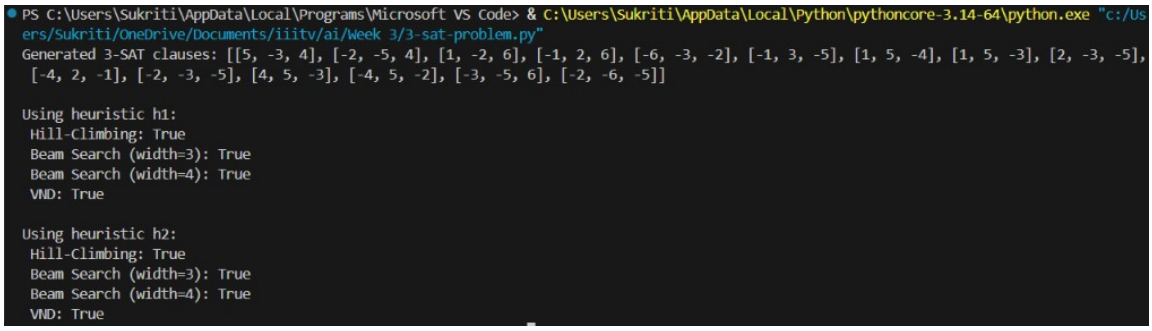


Figure 2: Output of 3-SAT Solver showing all algorithms returned True for both heuristics

Text output:

```
Generated 3-SAT clauses: [[5, -3, 4], [-2, -5, 4], [1, -2, 6], [-1, 2, 6], [-6, -3, -

Using heuristic h1:
 Hill-Climbing: True
 Beam Search (width=3): True
 Beam Search (width=4): True
 VND: True

Using heuristic h2:
 Hill-Climbing: True
 Beam Search (width=3): True
 Beam Search (width=4): True
 VND: True
```

V CODE AVAILABILITY

The complete source code for Week-3 is available at GitHub Repository:

`https://github.com/20261604007-ux/CS-605-Artifical-Intelligence`